# Aeza x Shopify

Product data, webhooks and testing guide

How the Aeza Partner app reads a brand's products, stock and orders from Shopify: the endpoints, webhooks and permissions it uses, what the data looks like, where every field is stored, and **step-by-step instructions to test all of it yourself** with Postman and the Neon SQL editor.

| Item | Detail |
|---|---|
| Example brand | Sands of India (`cb0c82-9b.myshopify.com`), the first brand connected |
| Shopify API | Admin GraphQL API, version `2026-07` |
| Aeza app | `https://aeza-partners.vercel.app` (repo `aeza-test-connector/`) |
| Database | Neon Postgres, schema `shopify_connector` |
| Files used in this guide | `docs/postman/aeza-shopify-admin.postman_collection.json` (Postman collection), `npm run store:token` (gets an API token) |

## Contents


1. Access: how we're allowed to read a brand's data
2. The Shopify calls we make (full sync, single product, stock)
3. Webhooks: how Shopify tells us about changes
4. Where each field is stored (database columns)
5. Fields we could add later
6. Testing guide, step by step (Tests 1-12)
7. Database checks (copy-paste SQL)
8. Troubleshooting

# 1. Access: how we're allowed to read a brand's data

| Item | Detail |
|---|---|
| API | Shopify **Admin GraphQL API**, version **2026-07**. No REST endpoints are used. |
| Endpoint | One URL for every call: `POST https://{shop}/admin/api/2026-07/graphql.json` |
| Auth header | `X-Shopify-Access-Token: <token>` |
| Token | One **offline access token per store**, issued when the brand installs its private Aeza app. Expires every **1 hour** and is renewed automatically with a refresh token (~90 days, renewed on use). Both are stored **encrypted** in `shopify_connector."Session"`. |
| Permissions | `read_products`, `read_inventory`, `read_locations`, `read_orders`, `write_orders`, `read_fulfillments` |
| Per-brand app | Each brand has its own private app (own client id/secret, stored encrypted in `ShopifyStore`), so a token only works for that one brand's store. |

# 2. The Shopify calls we make

## 2.1 Full catalog sync (on install + every night)

Large catalogs can't be paged through inside one web request, so we ask Shopify to export everything in the background (a **Bulk Operation**). Code: `app/services/product-sync.server.ts`.

1. **Start the export** with `bulkOperationRunQuery` (query below).
2. Shopify builds the file (Sands of India: 224 products in 4 seconds) and calls our `bulk_operations/finish` webhook.
3. We ask for the result: `node(id) { ... on BulkOperation { status errorCode url createdAt } }`.
4. We download `url`, a **JSONL** file (one JSON object per line), and save products in batches of 100. Products we have that weren't in the export are marked deleted, and `ShopifyStore.lastFullSyncAt` is set.

Start the export

```
mutation StartProductBulkSync($query: String!) {
  bulkOperationRunQuery(query: $query) {
    bulkOperation { id status }
    userErrors { field message }
  }
}

# $query:
{
  products {
    edges {
      node {
        id title handle descriptionHtml vendor productType tags status updatedAt
        media { edges { node { ... on MediaImage { image { url } } } } }
        variants {
          edges {
            node {
              id sku title price compareAtPrice inventoryQuantity
              selectedOptions { name value }
              inventoryItem { id }
            }
          }
        }
      }
    }
  }
}
```

In the JSONL file each product is one line, followed by one line per image and per variant, linked by `__parentId`. Real lines from Sands of India (description shortened):

JSONL export (first lines)

```
{"id":"gid://shopify/Product/9448278884632","title":"Sands Of India Women's White Yoke Embroidered Popli
    n Midi Dress - 1568","handle":"detailed-embroidered-yoke-and-tassels-dress","descriptionHtml":"<div
    class=\"container\">...","vendor":"Sands of India","productType":"Dress","tags":["SOIVODRS04/1569/15
    68"],"status":"ACTIVE","updatedAt":"2026-10-21T08:45:56Z"}
{"image":{"url":"https://cdn.shopify.com/s/files/1/0880/0554/2168/files/1568.png?v=1784615566"},"__paren
    tId":"gid://shopify/Product/9448278884632"}
{"image":{"url":"https://cdn.shopify.com/s/files/1/0880/0554/2168/files/SI-1811202401_1.jpg?v=1733288391
    "},"__parentId":"gid://shopify/Product/9448278884632"}
{"id":"gid://shopify/ProductVariant/...","sku":"SIDR0030324XS","title":"XS","price":"1908.00","compareAt
    Price":"4299.00","inventoryQuantity":10,"selectedOptions":[{"name":"Size","value":"XS"}],"inventoryI
    tem":{"id":"gid://shopify/InventoryItem/..."},"__parentId":"gid://shopify/Product/9448278884632"}
```

## 2.2 Single-product refresh (used by product webhooks)

When a product is created or edited we re-read it from Shopify (we don't trust the webhook body), with every variant:

Query

```
query RefreshProduct($id: ID!, $variantsAfter: String) {
  product(id: $id) {
    id title handle descriptionHtml vendor productType tags status updatedAt
    media(first: 50) { nodes { ... on MediaImage { image { url } } } }
    variants(first: 250, after: $variantsAfter) {
      nodes {
        id sku title price compareAtPrice inventoryQuantity
        selectedOptions { name value }
        inventoryItem { id }
      }
      pageInfo { hasNextPage endCursor }
    }
  }
}
```

Real response (Sands of India, shortened to 2 images / 2 variants)


```
{
  "data": {
    "product": {
      "id": "gid://shopify/Product/10025366651160",
      "title": "Sands Of India Black Poplin Embroidered Co-ord Set for Women - 3104",
      "handle": "soivocs76-black-poplin-embroidered-co-ord-set-for-women",
      "descriptionHtml": "<p data-path-to-node=\"2\"><b ...>Effortl...",
      "vendor": "Sands of India",
      "productType": "Co-ord set",
      "tags": ["3039"],
      "status": "ACTIVE",
      "updatedAt": "2026-10-22T07:49:36Z",
      "media": { "nodes": [
        { "image": { "url": "https://cdn.shopify.com/s/files/1/0880/0554/2168/files/3104.png?v=178298784
    7" } },
        { "image": { "url": "https://cdn.shopify.com/s/files/1/0880/0554/2168/files/SOIVOCT77B_2_dfa3cef
    4-eac2-4bab-957e-46feacc98fba.jpg?v=1767677901" } }
      ] },
      "variants": { "nodes": [
        { "id": "gid://shopify/ProductVariant/51615929434392", "sku": "SICO0410326XS", "title": "XS",
          "price": "2353.00", "compareAtPrice": "4999.00", "inventoryQuantity": 13,
          "selectedOptions": [{ "name": "Size", "value": "XS" }],
          "inventoryItem": { "id": "gid://shopify/InventoryItem/53675457118488" } },
        { "id": "gid://shopify/ProductVariant/51615929467160", "sku": "SICO04103260S", "title": "S",
          "price": "2353.00", "compareAtPrice": "4999.00", "inventoryQuantity": 23,
          "selectedOptions": [{ "name": "Size", "value": "S" }],
          "inventoryItem": { "id": "gid://shopify/InventoryItem/53675457151256" } }
      ], "pageInfo": { "hasNextPage": true, "endCursor": "eyJsYXN0X2lkIjo1MTYxNTkyOTQ2NzE2MCwibGFzdF92YW
    x1ZSI6Mn0=" } }
    }
  }
}
```


### 2.3 Stock refresh and shop currency

Stock for one inventory item (inventory webhook) / shop currency

```
query InventoryItemQuantity($id: ID!) {
  inventoryItem(id: $id) {
    variants(first: 1) { nodes { id inventoryQuantity } }
  }
}

{ shop { currencyCode } }   # read before creating an order
```

## 3. Webhooks: how Shopify tells us about changes

Subscriptions are declared per brand app in `shopify.app.<slug>.toml` (generated from `shopify.app.template.toml`) and pushed with `npx shopify app deploy --config <slug>`. Every delivery is:

- **verified** with that brand app's secret (HMAC signature),
- **recorded** in `WebhookEvent` by its `X-Shopify-Webhook-Id` (a repeat delivery is ignored),
- **answered** with HTTP 200 immediately, then **processed** in the background; the row ends as `done` or `failed` with the error.

| Topic | Sent to | What the app does |
|---|---|---|
| `products/create`, `products/update` | `/webhooks/events` | Re-fetch the product (2.2) and save product + variants |
| `products/delete` | `/webhooks/events` | Mark deleted, unpublish |
| `inventory_levels/update` | `/webhooks/events` | Re-read that item's stock, update `availableQty` |

| Topic | Sent to | What the app does |
|---|---|---|
| `orders/updated`, `orders/cancelled` | `/webhooks/events` | Update payment/fulfilment status and `cancelledAt` on **Aeza orders only** |
| `fulfillments/create`, `fulfillments/update` | `/webhooks/events` | Save tracking company / number / URL and shipment status |
| `bulk_operations/finish` | `/webhooks/events` | Import the full catalog export (2.1) |
| `app/uninstalled` | `/webhooks/app/uninstalled` | Store -> `uninstalled`, all products unpublished, sessions deleted |
| `app/scopes_update` | `/webhooks/app/scopes_update` | Update granted permissions |
| `customers/data_request`, `customers/redact`, `shop/redact` | `/webhooks/compliance` | GDPR; `shop/redact` deletes the shop's data |

Webhook bodies are Shopify's REST-style JSON (snake_case). The only fields we read:

| Topic | Fields used |
|---|---|
| products/create, products/update | `admin_graphql_api_id` |
| products/delete | `id` |
| inventory_levels/update | `inventory_item_id` |
| orders/updated, orders/cancelled | `admin_graphql_api_id`, `financial_status`, `fulfillment_status`, `cancelled_at` |
| fulfillments/create, fulfillments/update | `order_id`, `tracking_company`, `tracking_number`, `tracking_url`, `shipment_status` |
| bulk_operations/finish | `admin_graphql_api_id`, `type` |

Full payload examples for every topic: `https://shopify.dev/docs/api/webhooks/2026-07`. Real deliveries for a brand are visible in the Dev Dashboard -> the brand's app -> **Logs**.

# 4. Where each field is stored

## ShopifyProduct (one row per product)

| Column | Comes from | Example (Sands of India) |
|---|---|---|
| `shop` | the store | `cb0c82-9b.myshopify.com` |
| `productId` | `product.id` | `gid://shopify/Product/10025366651160` |
| `title` | `title` | Sands Of India Black Poplin Embroidered Co-ord Set for Women - 3104 |
| `handle` | `handle` | `soivocs76-black-poplin-embroidered-co-ord-set-for-women` |
| `descriptionHtml` | `descriptionHtml` | HTML |
| `vendor` | `vendor` | Sands of India |
| `productType` | `productType` | Co-ord set |
| `tags` | `tags` | `["3039"]` |
| `images` | every `media[].image.url` | JSON array of image URLs |
| `status` | `status` | `ACTIVE` / `DRAFT` / `ARCHIVED` |
| `shopifyUpdatedAt` | `updatedAt` | 2026-10-22 07:49:36 UTC |

| Column | Comes from | Example (Sands of India) |
|---|---|---|
| `published` | **ours**: the brand's choice in the Aeza dashboard (or "show all" mode) | true / false |
| `syncedAt`, `syncStatus`, `syncError`, `deletedAt`, `createdAt`, `updatedAt` | **ours** | |

A product is **live on Aeza** only when `published` is true, `status` is `ACTIVE` and `deletedAt` is empty.

### ShopifyVariant (one row per size / colour)

| Column | Comes from | Example |
|---|---|---|
| `variantId` | `variant.id` | `gid://shopify/ProductVariant/51615929434392` |
| `shop`, `productId` | parent product | |
| `sku` | `sku` | `SICO0410326XS` |
| `title` | `title` | XS |
| `options` | `selectedOptions` as an object | `{"Size": "XS"}` |
| `price` | `price` (selling price) | 2353.00 |
| `compareAtPrice` | `compareAtPrice` (MRP) | 4999.00 |
| `inventoryItemId` | `inventoryItem.id` | `gid://shopify/InventoryItem/53675457118488` |
| `availableQty` | `inventoryQuantity` (available, all locations) | 13 |

### ShopifyOrder (only orders Aeza created)

Filled by `POST /api/orders`, kept up to date by the order and fulfilment webhooks: `aezaOrderId`, `shopifyOrderId`, `shopifyOrderName`, `financialStatus`, `fulfillmentStatus`, `cancelledAt`, `trackingCompany`, `trackingNumber`, `trackingUrl`, `shipmentStatus`, `isTest`.

## 5. Fields we could add later

All of these exist in API 2026-07 (checked against Shopify's schema). Try them first in Postman request **A1** by adding them to the query.

| Level | Field | Gives you |
|---|---|---|
| Product | `options { name values }` | All option names/values (e.g. Size: XS...XL) |
| Product | `category { name fullName }` | Shopify's standard taxonomy category |
| Product | `createdAt`, `publishedAt` | Dates |
| Product | `totalInventory` | Stock across all variants |
| Product | `onlineStoreUrl` | Product page on the brand's own website |
| Product | `seo { title description }` | SEO title / description |
| Product | `featuredMedia { ... on MediaImage { image { url altText } } }` | Main image |
| Product | `metafields(first: 20) { nodes { namespace key value type } }` | Brand custom data (fabric, fit, care...) if they use it |
| Media | `image { altText width height }` | Image alt text and size |
| Variant | `barcode` | EAN / UPC |

| Level | Field | Gives you |
|---|---|---|
| Variant | `media(first: 5) { nodes { ... on MediaImage { image { url } } } }` | Variant-specific images (e.g. per colour) |
| Variant | `inventoryPolicy` | Whether the brand sells when out of stock |
| Variant | `inventoryItem { measurement { weight { value unit } } }` | Shipping weight |
| Variant | `position` | Size order |

To add one permanently: add the field to **both** queries in `app/services/product-sync.server.ts` (`BULK_PRODUCTS_QUERY` and `RefreshProduct`), add the column in `prisma/schema.prisma` with a new migration, and map it in `toSyncedProduct` / `toSyncedVariant` and `saveProducts`. The next full sync fills it for every product.

# 6. Testing guide, step by step

> **Read before testing.** Tests 1-5 and 7 only **read** data and are safe on Sands of India's live store. Tests 6, 8, 9 and 11 **change things in a Shopify store** (prices, stock, real orders, uninstalling). Run those on a **development store** you control, or only with the brand's agreement.

## 6.0 What you need (one-time setup)

| Need | How to get it |
|---|---|
| The repo on your computer | `aeza-test-connector/` with `npm install` done; Node.js 22.18 or newer (`node -v`) |
| `.env` in `aeza-test-connector/` | `DATABASE_URL`, `DIRECT_URL`, `ENCRYPTION_KEY` (same values as Vercel) |
| Postman | Desktop app from postman.com (free) |
| Neon access | console.neon.tech -> the project -> **SQL Editor** (for the database checks) |
| Shopify access | Login to the store's admin (to change prices/stock), and to dev.shopify.com (org **Aeza - public**) |
| For order tests only | `MAIN_BACKEND_API_KEY` set in Vercel (see Test 9) |
| For the nightly sync test only | `CRON_SECRET` set in Vercel (see Test 12) |

## Test 1: Get an API token

1. Open a terminal in `aeza-test-connector/`.
2. Run: `npm run store:token -- --shop cb0c82-9b.myshopify.com`
3. It prints a line "Access token for ... (valid until ...)" and then the token (starts with `shpca_`). Copy the token.

> The token can read that brand's products and orders for about an hour. Never paste it into chat, tickets, documents or a shared Postman workspace. If it stops working (HTTP 401), run the command again.

## Test 2: Set up Postman

1. Open Postman -> **Import** (top left) -> choose `docs/postman/aeza-shopify-admin.postman_collection.json`.
2. In the left sidebar click the collection **"Aeza - Shopify Admin API (brand store)"** -> **Variables** tab.
3. Check `shop` = `cb0c82-9b.myshopify.com` (change it for another brand).
4. Paste the token from Test 1 into the **Current value** of `access_token`. Click **Save**.
5. Leave the other variables as they are for now.

The collection has two folders: **A. Shopify Admin API** (reads directly from the brand's store with the token) and **B. Aeza app endpoints** (calls our own app). Folder A sends the `X-Shopify-Access-Token` header automatically.

## Test 3: Read one product (safe)

1. Open **A1. One product with all variants** -> **Send**.
2. Expected: HTTP **200** and JSON like section 2.2: title, handle, images, every size with SKU, price, MRP and stock.
3. Compare with our database: run the SQL below in Neon. Price, MRP and stock must match the Postman response.
4. Try another product: copy a product id from request A2 into the `product_id` variable and send again.

Database check (Neon SQL Editor)

```
SELECT p.title, v.title AS size, v.sku, v.price, v."compareAtPrice",
       v."availableQty", v."updatedAt"
FROM shopify_connector."ShopifyVariant" v
JOIN shopify_connector."ShopifyProduct" p
  ON p.shop = v.shop AND p."productId" = v."productId"
WHERE v.sku = 'SICO0410326XS';
```

## Test 4: Browse the catalog (safe)

1. Open **A2. Browse products** -> **Send**. You get 10 products, newest first.
2. For the next page: copy `pageInfo.endCursor` from the response, put it in the variables box as `{ "after": "<cursor>" }`, send again.
3. Check the total: run "Product counts" (section 7). Active + draft should equal the store's product count in Shopify admin.

## Test 5: Full catalog export (safe; triggers a re-import)

1. Open **A3. Start full catalog export** -> **Send**. Expected: `bulkOperation.status` = `CREATED`, `userErrors` empty.
2. Wait 10-30 seconds, then send **A4. Latest export status**. Repeat until `status` = `COMPLETED`.
3. Copy `url` from the response into a browser: it downloads the whole catalog as a `.jsonl` file (open it in any text editor).
4. Within a minute our app imports it. Run "Store status" (section 7): `lastFullSyncAt` must show the new time.
5. Run "Latest webhooks": the newest row is `BULK_OPERATIONS_FINISH` with status `done`.

## Test 6: Live updates from Shopify (changes the store)

Use a development store, or agree a test product with the brand. For each change, wait up to one minute, then check.

| Do this in Shopify admin | Then check (section 7) | Expected |
|---|---|---|
| Change one size's **price** | "One SKU" with that SKU | `price` updated; `updatedAt` = just now |
| Set one size's **stock** to 0 | "One SKU" | `availableQty` = 0 |
| Edit a product **title** or add an **image** | "Recently changed sizes" | Product appears at the top |
| Change a product from **Draft to Active** | "Product counts" | Active count +1 |
| **Create** a new product | "Product counts" | Total +1; in "Only selected" mode it starts unpublished |
| **Delete** a product | "Latest webhooks" | `PRODUCTS_DELETE` done; product marked deleted, unpublished |

Each change also adds a row in "Latest webhooks" (`PRODUCTS_UPDATE`, `INVENTORY_LEVELS_UPDATE`, ...) with status `done`. A `failed` row shows the reason in the `error` column.

## Test 7: Publish choices in the Aeza dashboard (safe)

1. Shopify admin -> **Apps** -> **Aeza x sandsofindia** opens the Aeza dashboard.
2. **Only selected products**: tick a product -> **Save changes**. "Product counts" -> published +1.
3. **Show all products on Aeza**: every active product becomes published, drafts become unpublished. "Products live on Aeza" = number of active products.
4. Switch back to **Only selected products** if that's what the brand wants (current choices are kept).

> This changes what the brand shows on Aeza. On a live brand, only do it with their agreement.

## Test 8: Uninstall and reinstall (development store only)

1. Shopify admin -> **Settings -> Apps** -> the Aeza app -> **Uninstall**.
2. "Store status": `status` = `uninstalled`. "Products live on Aeza" = 0. "Token is stored encrypted": no row for that shop.
3. Reinstall with the install link -> `status` = `connected` again and a fresh full sync runs (`lastFullSyncAt` updates).

## Test 9: Create an order through the Aeza API (development store recommended)

**One-time setup:** generate a key (`openssl rand -hex 32`), add it in Vercel -> Settings -> Environment Variables as `MAIN_BACKEND_API_KEY` (Production, Sensitive), then **redeploy**. Put the same value in the Postman variable `main_backend_api_key`.

1. Send **B2. Order API without key**. Expected: **401** `{"error":"unauthorized"}`. Proves the endpoint is protected.
2. In the variables: set `variant_id` to a size that is **live on Aeza** and in stock (from "One SKU": the `variantId` column, or request A1), and `aeza_order_id` to a new value, e.g. `AEZA-TEST-0002`.
3. Open **B1. Create test order** -> check the Body (customer, address, price) -> **Send**.
4. Expected: **201** `{ "shopifyOrderId": "gid://shopify/Order/...", "shopifyOrderName": "#1234" }`.
5. In Shopify admin -> **Orders**: the order is **Paid**, tagged **Aeza Test**, note contains the Aeza order id, and that size's stock went down by 1.
6. Send **B1 again with the same** `aeza_order_id`. Expected: **200** with `"duplicate": true` and the same order. No second order in Shopify.
7. "Aeza orders" (section 7): one row, `financialStatus` = `paid`, `isTest` = true.

| Response | Meaning |
|---|---|
| 201 | Order created |
| 200 + duplicate: true | This Aeza order id was already created; nothing new |
| 400 invalid_request | Body is missing or wrong; `message` says which field |
| 401 unauthorized | Missing or wrong `MAIN_BACKEND_API_KEY` |
| 404 unknown_shop / 409 store_not_connected | Store not registered / not installed |
| 409 order_in_progress | Same id is being created right now; retry in a few seconds |
| 422 unknown_variant / product_not_available | Variant not in our data / product not live on Aeza |
| 422 out_of_stock / shopify_rejected | Shopify refused it; see `details` |
| 502 shopify_unavailable | Outcome unknown; the id is locked 2 minutes. Check Shopify before retrying |

> Every successful B1 creates a **real, paid order** in the store (tagged "Aeza Test"). Afterwards cancel it in Shopify admin (Orders -> the order -> More actions -> Cancel order, restock items). That also tests Test 10's cancel step.

## Test 10: Shipping, tracking and cancellation (development store)

1. In Shopify admin open the test order -> **Fulfill items** -> add a tracking number and carrier -> **Fulfill**.

2. Within a minute, "Aeza orders": `trackingCompany`, `trackingNumber`, `trackingUrl` filled; `fulfillmentStatus` = `fulfilled`.
3. Cancel the order (More actions -> Cancel order). "Aeza orders": `cancelledAt` is set.
4. Orders the brand creates themselves (not via Aeza) are ignored: they never appear in `ShopifyOrder`.

### Test 11: Duplicate webhooks are processed once (safe)

1. Run "Webhook summary" (section 7). Each delivery appears exactly once, because `webhookId` is the table's primary key and repeats are skipped before any work is done.
2. Every row should be `done`. Any `failed` row: run "Latest webhooks" and read the `error` column.
3. Optional: in the Dev Dashboard -> the app -> **Logs**, compare deliveries with the table; every delivery should show response 200.

### Test 12: Nightly re-sync (safe)

**One-time setup:** generate a secret (`openssl rand -hex 32`), add it in Vercel as `CRON_SECRET` (Production, Sensitive), redeploy. Vercel then calls the sync automatically every night at 02:00 IST (`vercel.json`). Put the same value in the Postman variable `cron_secret` to run it by hand.

1. Send **B3. Run nightly re-sync now**. Expected: **200** `{ "started": 1, "failed": [], "prunedWebhookEvents": 0 }` (`started` = number of connected stores).
2. Without the secret it returns **401**.
3. After a minute, "Store status": `lastFullSyncAt` = just now.

# 7. Database checks (copy-paste SQL)

Neon console (console.neon.tech) -> the project -> **SQL Editor** -> choose the production branch and database `neondb` -> paste a query -> **Run**. These are all read-only. For another brand, replace `cb0c82-9b.myshopify.com`.

**Store status**

```
SELECT shop, status, "publishMode", "installedAt", "lastFullSyncAt"
FROM shopify_connector."ShopifyStore";
```

**Token is stored encrypted**

```
SELECT shop, "isOnline", scope, expires,
       left("accessToken", 7) AS token_prefix   -- must be enc:v1:
FROM shopify_connector."Session";
```

**Product counts**

```
SELECT status, count(*) AS products,
       count(*) FILTER (WHERE published) AS published
FROM shopify_connector."ShopifyProduct"
WHERE shop = 'cb0c82-9b.myshopify.com'
GROUP BY status;
```

**Products live on Aeza**

```
SELECT count(*) AS live_on_aeza
FROM shopify_connector."ShopifyProduct"
WHERE shop = 'cb0c82-9b.myshopify.com'
  AND published AND status = 'ACTIVE' AND "deletedAt" IS NULL;
```

**One SKU (price, MRP, stock)**

```
SELECT p.title, v.title AS size, v.sku, v.price, v."compareAtPrice",
       v."availableQty", v."updatedAt"
FROM shopify_connector."ShopifyVariant" v
JOIN shopify_connector."ShopifyProduct" p
  ON p.shop = v.shop AND p."productId" = v."productId"
WHERE v.sku = 'SICO0410326XS';
```

**Recently changed sizes**

```
SELECT p.title, v.title AS size, v.sku, v.price, v."availableQty", v."updatedAt"
FROM shopify_connector."ShopifyVariant" v
JOIN shopify_connector."ShopifyProduct" p
  ON p.shop = v.shop AND p."productId" = v."productId"
WHERE v.shop = 'cb0c82-9b.myshopify.com'
ORDER BY v."updatedAt" DESC
LIMIT 10;
```

**Latest webhooks**

```
SELECT topic, status, error, "receivedAt", "processedAt"
FROM shopify_connector."WebhookEvent"
WHERE shop = 'cb0c82-9b.myshopify.com'
ORDER BY "receivedAt" DESC
LIMIT 20;
```

**Webhook summary**

```
SELECT topic, status, count(*)
FROM shopify_connector."WebhookEvent"
WHERE shop = 'cb0c82-9b.myshopify.com'
GROUP BY topic, status
ORDER BY topic;
```

**Aeza orders**

```
SELECT "aezaOrderId", "shopifyOrderName", "financialStatus", "fulfillmentStatus",
       "cancelledAt", "trackingCompany", "trackingNumber", "trackingUrl",
       "shipmentStatus", "isTest", "createdAt"
FROM shopify_connector."ShopifyOrder"
ORDER BY "createdAt" DESC;
```

# 8. Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| Postman returns **401** on folder A | Token expired (they last ~1 hour) | Run `npm run store:token` again, update `access_token` |
| Postman returns HTML instead of JSON | Temporary Shopify error, or wrong `shop` | Check `shop`; send again |
| App shows "store isn't set up for Aeza" (403) | Store not registered | `npm run store:add` for that store |
| Installed, but 0 products | Webhooks not pushed to that app | `npx shopify app deploy --config <slug>`, then Test 5 |
| A webhook row is `failed` | Error during processing | Read `error` in "Latest webhooks"; the nightly sync repairs product data |
| `store:token` / scripts can't reach the database | Network or Neon compute paused | Check Neon console; retry; try another network |
| Order API **401** | `MAIN_BACKEND_API_KEY` not set / not redeployed / wrong value | Set it in Vercel, redeploy, update Postman |
| Order **422 product_not_available** | Product not published, not Active, or deleted | Publish it in the Aeza dashboard / make it Active |
| Order **422 out_of_stock** | Not enough stock for that size | Choose a size with stock |
| Nightly sync **401** | `CRON_SECRET` not set or different | Set it in Vercel, redeploy |

Code references: `app/services/product-sync.server.ts` (sync), `app/routes/webhooks.events.tsx` (webhooks), `app/services/orders.server.ts` (orders), `app/shopify.server.ts` (per-store auth), `README.md` (env vars, adding a brand).